

Prontuario FI2 — da tenere aperto all'esame

Cos'è. Materiale di consultazione per la prova pratica, dove si possono portare tutti i file che si vuole. Non spiega: *risponde*. Per il perché, ogni sezione rimanda alla guida o agli appunti del modulo.

Come si usa in prova. 1. Hai un **errore rosso in Eclipse** o un'eccezione nella console → Ctrl+F sul testo del messaggio: ogni sezione si apre con la tabella *errore* → *causa* → *rimedio*, con il testo esatto sia di Eclipse sia di javac. 2. Devi **fare una cosa** (convertire, arrotondare, leggere un numero da stringa) → *Indice per bisogno*. 3. ▲ segna le righe dove Lorenzo ha già sbagliato almeno una volta: lì rallenta.

Come cresce. Una sezione per esercitazione, scritta **dopo** averla fatta, e aggiornata a ogni fine sessione (/chiudi, passo 8): codice nuovo imparato, ingegni, frammenti da copiare. Contiene solo ciò che è stato **incontrato davvero** — gli schemi delle prove d'esame entrano quando si fanno le prove, non prima. **Ogni frammento di codice è commentato riga per riga**, così si usa e si interpreta senza rileggere la teoria.

Fonti. [sl. N] = pagina del PDF di slide della sezione. [oltre la fonte] = non è nelle slide, verificato in jshell (JDK 21) e, per i messaggi di Eclipse, con il suo compilatore (ecj 3.38), il 2026-09-26.

Indice per bisogno

Devo...	Vai a
capire un errore «Type mismatch» / «lossy conversion»	02x · A
sapere se un assegnamento fra numeri compila	02x · C
capire perché un conto dà un valore assurdo (overflow, Infinity, NaN)	02x · D
confrontare due double	02x · D
arrotondare a intero	02x · E
trasformare una stringa in numero, o un numero in stringa	02x · F
fare conti su caratteri ('7' → 7, lettera successiva)	02x · G
scrivere un main e leggere gli argomenti	02x · H
sapere quanto è grande / fin dove arriva un tipo	02x · B

02x — Tipi base & dintorni

Fonti: materiali/slide/02x-x1-Esercitazione Tipi base.pdf (42 pagine), 02z-Addendum-Main in Java21.pdf (7 pagine). Spiegazioni: lezioni/guida_lab_02x_tipi_base.md §1-§6.

A — Errore → causa → rimedio

Il testo di Eclipse è quello che vedi all'esame; quello di javac è quello di jshell e del terminale. **Stessa causa, parole diverse.**

Eclipse	javac / jshell	Causa	Rimedio
Type mismatch: cannot convert from double to float	incompatible types: possible lossy conversion from double to float	letterale senza F (3.54 è double), o espressione con un double dentro	3.54F, oppure cast (float) espr — $\triangle$ vedi C
Type mismatch: cannot convert from long to int	... possible lossy conversion from long to int	letterale con L; oppure Math.round(double), che restituisce long	(int) Math.round(x)
Type mismatch: cannot convert from double to int	... lossy conversion from double to int	letterale con punto (3.0); oppure Math rint, sqrt, pow, che restituiscono double	(int) espr — ma tronca: vedi E
Type mismatch: cannot convert from int to short / to byte	... lossy conversion from int to short	aritmetica su byte/short produce int (c + 1); oppure si assegna una <i>variabile</i> int	(short)(c + 1) — parentesi attorno all'intera espressione
Type mismatch: cannot convert from int to char	... lossy conversion from int to char	ch + 1 è un int	(char)(ch + 1), oppure ch++ (compila: ha il cast incorporato)
Type mismatch: cannot convert from String to int	String cannot be converted to int	gli argomenti del main sono stringhe, non numeri	Integer.parseInt(s) / Double.parseDouble(s) — vedi F
The literal 3000000000 of type int is out of range	integer number too large	un letterale senza L è int, e non arriva a $3 \cdot 10^9$	3000000000L

Eccezioni a run-time (compila tutto, esplode eseguendo — in console, in rosso):

Eccezione	Quando	Rimedio
ArithmeticException: / by zero	divisione o % fra interi per zero. Fra reali non succede: dà Infinity/NaN [sl. 21-22]	controllare il divisore prima
NumberFormatException: For input string: "..."	parseInt/parseDouble su stringa non valida — le virgolette nel messaggio mostrano la stringa esatta : guardale per vedere spazi e virgole	controllare la stringa prima di convertirla [sl. 35] — vedi F

Errori silenziosi — compila, gira, e il risultato è sbagliato. Nessun messaggio.

Sintomo	Causa probabile	Vai a
numero enorme negativo, o segno invertito	overflow	D
x == y falso fra due double che «dovrebbero» essere uguali	errore numerico	D
(int) 3.9 dà 3	il cast tronca , non arrotonda	E
Math rint(2.5) dà 2.0	rint arrotonda al pari	E

B — I tipi primitivi

Tipo	Byte	Range	Letterale	Note
byte	1	−128 ... 127	—	[sl. 11]
short	2	−32 768 ... 32 767	—	[sl. 11]
char	2	0 ... 65 535 (senza segno)	'A' apici singoli	Unicode, UTF-16 [sl. 23]
int	4	$\approx \pm 2,1 \cdot 10^9$ (Integer.MAX_VALUE = 2 147 483 647)	42	default dei letterali interi [sl. 11]
long	8	$\approx \pm 9,2 \cdot 10^{18}$	42L	«le costanti long terminano con la lettera L» [sl. 11]

Tipo	Byte	Range	Letterale	Note
float	4	fino a $\approx 3,4 \cdot 10^{38}$	3.54F	6-7 cifre significative [sl. 14]
double	8	fino a $\approx 1,8 \cdot 10^{308}$	3.54	14-15 cifre significative. Default dei letterali con punto [sl. 14]
boolean	—	true / false	—	tipo primitivo [sl. 27]

Il letterale ha un tipo, e lo decide la **sintassi**, non il valore: 42 → int, 42L → long, 3.54 → double, 3.54F → float, 'A' → char, "A" → String.

C — Conversioni fra numerici: compila o no?

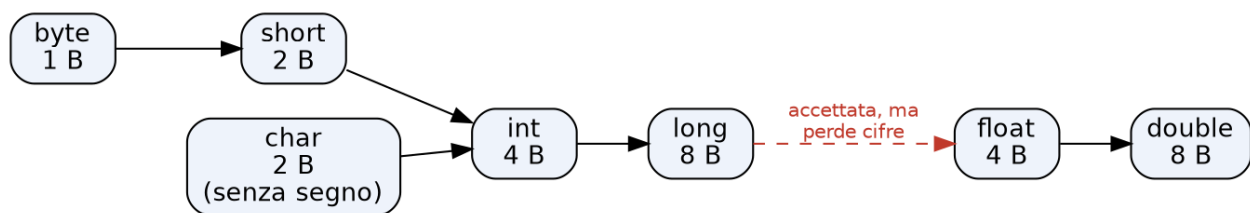


Figura 1: Conversioni allarganti: ogni freccia (e ogni cammino di frecce) è accettata senza cast

Si legge nel verso delle frecce: se da X a Y c'è un cammino, $Y \text{ var} = \langle X \rangle$ compila senza cast. Contro il verso serve il cast. La freccia tratteggiata è l'unico punto dove «compila» e «non perde niente» smettono di coincidere (riga 5 sotto). Intercetta l'errore del verso invertito — $\triangle$ trasversale n. 5.

Il procedimento in 2 passi — sempre in quest'ordine: 1. **Tipo del lato destro:** letterale → tabella B; espressione → il tipo **più largo** fra gli operandi, e **mai meno di int** (short + int è int, ma anche byte + byte). 2. **Cammino nel diagramma** dal tipo di destra a quello di sinistra? Sì → compila. No → cast.

«In Java, C#, Scala sono ammessi solo gli assegnamenti che non causano perdita di informazione.» double x = 3.54F; lecita, float f = 3.54; illecita. [sl. 15]

Eccezioni al procedimento, tutte provate nel drill:

#	Caso	Esito	Perché
1	short s = 300; char c = 65; byte b = 10;	✓ compila	costante nota al compilatore e ci sta nel tipo
2	byte b = 300;	✗	costante, ma non ci sta
3	short s = 40000; int q = 10; byte b = q;	✗	q è una variabile: il compilatore non ne segue il valore
4	byte b = 127; b++;	✓ compila → b = -128	++ ha il cast incorporato: avvolge senza avvisare [sl. 13]
5	float h = 123456789L;	✓ compila → stampa 1.2345679E8	long → float è ammessa (conta il <i>range</i>), ma float tiene solo 7 cifre

Il cast — (tipo) espressione: è la dichiarazione di **Design Intent**, cioè la perdita è una scelta tua e ne rispondi tu. ⚠ Usa questo termine all'orale (trasversale n. 4).

```
float f = (float) 3.54;           // double → float: perdita dichiarata, compila
int t = (int) 3.9;               // t = 3: il cast TRONCA, non arrotonda
int u = (int) -3.9;              // u = -3: tronca verso lo zero, non verso il basso
short d = (short)(c + 1);        // parentesi esterne obbligatorie: c + 1 è un int
// short d = (short) c + 1;      // SBAGLIATO: il cast lega solo a c, la somma torna
```

D — Aritmetica: le trappole che compilano

Overflow — Java protegge dalle *conversioni* che perdono dati, **non** dall'overflow aritmetico: il valore avvolge in silenzio [sl. 12-13].

Espressione	Valore
Integer.MAX_VALUE + 1	-2147483648
byte b = 125; b++; ++b; ++b;	-128 (da 127 passa a -128)
short e = c + 1; con c short	✗ non compila — qui c'è una conversione int → short, e il compilatore la ferma [sl. 12]

Reali e IEEE-754 — «Lo standard IEEE-754 incorpora le nozioni di infinito e Not-A-Number (NaN)» [sl. 21]; «I tipi interi non seguono lo standard IEEE-754» [sl. 22].

Espressione	Valore
5.0 / 0	Infinity

Espressione	Valore
-5.0 / 0	-Infinity
0.0 / 0.0 · Math.sqrt(-1)	NaN
0.0 / -5	-0.0 — «Java mantiene il segno nel risultato, MA ciò non ne altera la semantica» [sl. 20]: -0.0 == 0.0 è true
5 / 0.5 % 0	ArithmeticException: / by zero
Double.NaN == Double.NaN	false — per testare usa Double.isNaN(x) [oltre la fonte]

Confrontare due double: mai con ==. Il collaudo delle equazioni avverte «soluzioni coincidenti – occhio agli errori numerici» [sl. 39]: il discriminante che «dovrebbe» valere 0 può valere 1e-16.

```
// 0.1 + 0.2 == 0.3 → false    (0.1 + 0.2 vale 0.30000000000000004)
final double EPS = 1e-9;      // tolleranza: quanto vicino basta per dire '
if (Math.abs(delta) < EPS) {   // delta "è zero" se sta entro la tolleranza
    // radici coincidenti
}
```

E — Arrotondare

Funzione	Restituisce	2.5	-2.5	2.7	Per assegnarla a int
(int) x	int	2	-2	2	— (tronca verso zero)
Math.round(x)	long (da double)	3	-2	3	(int) Math.round(x)
Math rint(x)	double	2.0	-2.0	3.0	(int) Math.rint(x)

- [sl. 19]: «rint arrotonda un valore reale all'intero più vicino (risultato reale); round arrotonda un valore reale all'intero più vicino (risultato intero)». Sui valori **esattamente a metà** divergono: rint va al **pari** (0.5→0, 1.5→2, 2.5→2, 3.5→4), round va verso $+\infty$ (2.5→3, -2.5→-2). [oltre la fonte]
- Altre di Math [sl. 10, 19]: sqrt, pow(x, y) (→ double: pow(2,10) = 1024.0), sin, hypot(x, y) = $\sqrt{x^2+y^2}$ «senza errori di overflow/underflow intermedi», log1p(p) = $\ln(1+p)$, la costante Math.PI.

F — Stringa ↔ numero

Stringa → numero [sl. 34]: Integer.parseInt, Double.parseDouble. «MA questi metodi pretendono stringhe “giuste”! altrimenti... BOOM!» [sl. 34]

Input	Integer.parseInt	Double.parseDouble
"42"	42	42.0
"3.54"	X NumberFormatException	3.54
"3,54" (virgola)	X	X — il separatore è il punto
"aa"	X	X

In Java «per evitare tale errore si può solo controllare la stringa prima di convertirla» [sl. 35].

```
double a = Double.parseDouble(args[0]); // args[0] è una String: va convertita
// con args[0] = "aa" qui il programma si ferma con NumberFormatException
```

Numero → stringa

```
String s1 = String.valueOf(3.0); // "3.0" — funzione statica [sl. 23]
String s2 = "" + 3.0;           // "3.0" — la concatenazione converte da solo
String s3 = String.valueOf('A'); // "A" — anche da char
```

Concatenazione con +: «l'operatore + concatena stringhe e nel farlo converte anche in stringa ciò che stringa non è» [sl. 7]. Si valuta **da sinistra**, e finché non compare una stringa è somma:

Espressione	Valore
"a" + 1 + 2	"a12"
1 + 2 + "a"	"3a"
"x" + 'a' + 'b'	"xab"
'a' + 'b' + "x"	"195x" — due char sommati sono un int (97 + 98)

G — Caratteri

char è un numero da 16 bit con segno di carattere: si somma, si confronta, si converte.

Espressione	Valore	Uso
(int) 'A' · int m = 'A';	65	codice del carattere [sl. 23]
char c = 65;	'A'	costante che ci sta → nessun cast (C, riga 1)
'7' - '0'	7	cifra-carattere → valore
'a' + 1	98 (int!)	—
(char)('a' + 1)	'b'	lettera successiva
ch++	'b'	compila senza cast (C, riga 4)

Character, classe-libreria — «non si può chiedere a un valore primitivo di “fare qualcosa”... bisogna chiamare una qualche funzione di una qualche libreria» [sl. 27]:

Chiamata	Valore	Nota
Character.toUpperCase('a') / toLowerCase		[sl. 28]
Character.isWhitespace('true')		spazio, tab, a capo... [sl. 28]
Character.digit('8', 10) · digit('8', 7)	8 · -1	[sl. 29]
Character.digit('B', 16) · digit('B', 10)	11 · -1	-1 = «segnale di errore»: va controllato [sl. 29]

Integer [sl. 30]: signum(-42) → -1; rotateLeft(6, 1) → 12; rotateRight(6, 2) → -2147483647 (i bit usciti a destra rientrano a sinistra: è una *rotazione*, non uno shift).

Unicode, UTF, e i tre numeri di un carattere [sl. 23-26]. «In Java, i caratteri sono Unicode (16 bit, UTF-16)»; i byte UTF si ottengono «partendo da una stringa, NON da un carattere», con `getBytes(...)`. $\triangle$ char e byte non stanno sullo stesso piano (occorrenza del 16/09).

Per U+1F608 (faccina, fuori dal piano base)	Valore	Risponde a
<code>s.length()</code>	2	quanti char (unità UTF-16: coppia surrogata)
<code>s.getBytes("UTF-8").length</code>	4	quanto occupa in UTF-8
<code>s.codePointCount(0, s.length())</code>	1	quanti caratteri veri
<code>"À".getBytes("UTF-8").length</code>	2	—

BOM di UTF-16. `"A".getBytes("UTF-16")` → [-2, -1, 0, 65] = FE FF 00 41. I due byte in testa sono il BOM U+FEFF. Messi nell'ordine FE FF, indicano **big-endian**: è la sl. 25 a essere corretta («Everything in Java is stored in big-endian order»), mentre la didascalia della sl. 24 («FE, FF è il marcatore little endian») è **imprecisa**. Verificato in jshell il 2026-09-26. UTF-8 → [65], UTF-32 → [0, 0, 0, 65].

H — main e argomenti

```
public class Prog {
    public static void main(String[] args) {
        if (args.length == 0)
            System.out.println("Nessun argomento");
        for (int i = 0; i < args.length; i++)
            System.out.println("argomento " + i + ": " + args[i]);
    }
}
```

// in Java il main sta sempre
// static: esiste per tutto il
// length è una PROPRIETÀ dell'
// da 0: args[0] è il primo arg
// + converte

- static «perché deve esistere dall'inizio alla fine del programma»; public «perché deve essere visibile dall'esterno» [sl. 2]. In C `argv[0]` è il nome del programma, in Java no [sl. 3].
- `java Prog alfa "beta gamma"` → 2 argomenti: le virgolette tengono insieme [sl. 9].

- Gli argomenti sono **sempre String**: per i numeri serve `parseDouble` — vedi F [sl. 37].
- Java 21 (preview) accetta anche `void main()` senza classe né argomenti, compilando con `javac --enable-preview --source 21`. Ordine di ricerca: statico con argomenti → statico senza → d'istanza con argomenti → d'istanza senza [02z, sl. 7]. All'esame usa la forma classica.

I — Per l'orale: Java e gli altri

	Java	C#	Scala / Kotlin
tipi base	primitivi , non oggetti	classi trattate come primitivi (<code>int = Int32</code>)	veri tipi-oggetto
chiedere un servizio	funzione statica di libreria: <code>Character.toUpperCase(c)</code>	anche metodo: <code>ch.ToString()</code> , <code>'B'.ToString()</code>	metodo: <code>x.toFloat</code> (Scala senza parentesi), <code>x.toFloat()</code> (Kotlin)
stringa → numero	<code>Integer.parseInt</code>	<code>Convert.ToInt32</code>	<code>toInt</code> ; senza eccezione <code>toDoubleOption</code> / <code>toDoubleOrNull</code>
main	in una classe, <code>public static</code>	in una classe	Scala in un <code>object</code> ; Kotlin a livello di file

[sl. 2, 27, 31-35]. ⚠ `Integer`, `Character` di Java «NON VANNO CONFUSE COI TIPI-OGGETTO DI Scala e Kotlin» [sl. 27]: sono librerie **accanto** al primitivo, non al suo posto.